UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Swarm Runner Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Swarm Runner

Swarm Runner is a standalone editable Unity game template. Drag left and right to steer your running blob crowd. Choose gates that change crowd size, collect coins and avoid traps. Clashes and the Jelly King use your crowd strength. Local unit and income upgrades unlock as progress advances.

Includes 25 curated levels, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Swarm Runner > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/CrowdClash/Scenes/CrowdClash_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/CrowdClash`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/crowdclash contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/CCArt.cs`
- `Scripts/CCCrowd.cs`
- `Scripts/CCLevels.cs`
- `Scripts/CCMeshes.cs`
- `Scripts/CCWorld.cs`
- `Scripts/CrowdClashGame.cs`

Main flow: CrowdClashGame.cs. UI and popup consumers: CrowdClashGame.cs. Model/generator/loading: CCLevels.cs. Authoring: Editor/CrowdClashLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Drag left and right to steer your running blob crowd. Choose gates that change crowd size, collect coins and avoid traps. Clashes and the Jelly King use your crowd strength. Local unit and income upgrades unlock as progress advances.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact names/unlocks/free charges: Scripts/CrowdClashGame.cs; effect/help strings: Scripts/CrowdClashGame.cs.

Helper	Unlock	Free charges	Action and effect
Shield	5	2	A jelly bubble protects your crowd from traps for a few seconds.
Magnet	8	2	Pulls every coin nearby into your crowd.
Double Up	11	2	The next gate you pass gives twice the blobs.
Frenzy	15	2	Your blobs hit twice as hard in clashes and against the Jelly King.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Swarm Runner > Level Editor**. Exact implementation: `Editor/CrowdClashLevelEditor.cs`.

Open the Swarm Runner editor and select an early level. Use Dup or Duplicate before editing. Change its visible name and one rule or board element with the exact controls below. Save explicitly and run the current entry through the editor validation and play controls. The per-game source-control table includes the current brush, geometry, target, rule and AI fields; do not treat hierarchy or resource IDs as display text.

1. Back up Resources/crowdclash/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`CrowdClashLevelEditor.cs:42`	`[MenuItem("Szekipapa77/Swarm Runner/Level Editor")]`
`CrowdClashLevelEditor.cs:144`	`EditorGUILayout.HelpBox("Editing is disabled in Play mode. Stop Play mode to edit levels (the editor works entirely in Edit mode).", MessageType.Warning);`
`CrowdClashLevelEditor.cs:153`	`if (GUILayout.Button("Save", EditorStyles.toolbarButton, GUILayout.Width(60))) Save();`
`CrowdClashLevelEditor.cs:154`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60))) { if (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel")) Load(); }`
`CrowdClashLevelEditor.cs:155`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(90))) ValidateAll(true);`

`CrowdClashLevelEditor.cs:177`	`if (GUILayout.Button("Add")) { var d = new CCLevelData { name = "New track", theme = CCGen.ThemeFor(_file.levels.Count + 1) }; d.items.Add(new CCItem { type = "gate", z = 12, ops = "+10|x2" }); _file.levels.Insert(Mathf.Min(_sel + 1, _file.levels.Count), d); _sel = Mathf.Min(_sel + 1, _file.levels.Count - 1); _item = -1; ValidateAll(false); MarkDirty(true); }`
`CrowdClashLevelEditor.cs:178`	`if (GUILayout.Button("Dup") && Cur != null) { _file.levels.Insert(_sel + 1, Cur.Clone()); _sel++; ValidateAll(false); MarkDirty(true); }`
`CrowdClashLevelEditor.cs:179`	`if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete", "Delete level " + (_sel + 1) + "?", "Delete", "Cancel")) { _file.levels.RemoveAt(_sel); _sel = Mathf.Clamp(_sel - 1, 0, _file.levels.Count - 1); _item = -1; ValidateAll(false); MarkDirty(true); }`
`CrowdClashLevelEditor.cs:180`	`if (GUILayout.Button("Up") && _sel > 0) { (_file.levels[_sel], _file.levels[_sel - 1]) = (_file.levels[_sel - 1], _file.levels[_sel]); _sel--; ValidateAll(false); MarkDirty(true); }`
`CrowdClashLevelEditor.cs:181`	`if (GUILayout.Button("Dn") && _sel < _file.levels.Count - 1) { (_file.levels[_sel], _file.levels[_sel + 1]) = (_file.levels[_sel + 1], _file.levels[_sel]); _sel++; ValidateAll(false); MarkDirty(true); }`
`CrowdClashLevelEditor.cs:418`	`EditorGUILayout.LabelField("Level " + (_sel + 1), EditorStyles.boldLabel);`
`CrowdClashLevelEditor.cs:420`	`d.name = EditorGUILayout.TextField("Name", d.name);`
`CrowdClashLevelEditor.cs:422`	`d.theme = EditorGUILayout.Popup("Theme", Mathf.Clamp(d.theme, 0, names.Length - 1), names);`
`CrowdClashLevelEditor.cs:423`	`d.width = EditorGUILayout.Slider("Track half width", d.width, 3.2f, 4.8f);`
`CrowdClashLevelEditor.cs:424`	`int fi = EditorGUILayout.Popup("Finish", d.IsBoss ? 1 : 0, Finishes); d.finish = Finishes[fi];`

CrowdClashLevelEditor.cs:425	`if (d.IsBoss) d.bossHp = EditorGUILayout.IntField("Jelly King HP", d.bossHp);`
CrowdClashLevelEditor.cs:426	`else d.perStep = EditorGUILayout.IntSlider("Blobs per step", d.perStep, 1, 40);`
CrowdClashLevelEditor.cs:427	`d.hint = EditorGUILayout.TextField("Start hint", d.hint);`
CrowdClashLevelEditor.cs:432	`if (GUILayout.Button("Fit finish to crowd")) FitFinish(d);`
CrowdClashLevelEditor.cs:433	`if (GUILayout.Button("Auto-space items")) AutoSpace(d);`
CrowdClashLevelEditor.cs:438	`if (GUILayout.Button("+Gate")) AddItem(d, new CCItem { type = "gate", z = endZ + 15, ops = "+10|x2" });`
CrowdClashLevelEditor.cs:439	`if (GUILayout.Button("+Squad")) AddItem(d, new CCItem { type = "enemy", z = endZ + 16, count = 10 });`
CrowdClashLevelEditor.cs:440	`if (GUILayout.Button("+Obstacle")) AddItem(d, new CCItem { type = "obstacle", z = endZ + 12, ob = "spike" });`
CrowdClashLevelEditor.cs:441	`if (GUILayout.Button("+Coins")) AddItem(d, new CCItem { type = "coins", z = endZ + 6, count = 6 });`
CrowdClashLevelEditor.cs:450	`if (GUILayout.Button("Validate", GUILayout.Height(26))) { var v = ValidateOne(_sel); ShowNotification(new GUIContent(v.Ok ? "Level OK" : "FAIL: " + v.Why)); }`
CrowdClashLevelEditor.cs:451	`if (GUILayout.Button("Play this level", GUILayout.Height(26))) PlayLevel(_sel + 1);`
CrowdClashLevelEditor.cs:462	`if (_item < 0 || _item >= d.items.Count) { EditorGUILayout.HelpBox("Select an item on the track to edit it.", MessageType.None); return; }`
CrowdClashLevelEditor.cs:464	`EditorGUILayout.LabelField("Selected item", EditorStyles.boldLabel);`

CrowdClashLevelEditor.cs:466	int ti = Mathf.Max(0, Array.IndexOf(Types, it.type)); ti = EditorGUILayout.Popup("Type", ti, Types); it.type = Types[ti];
CrowdClashLevelEditor.cs:467	it.z = EditorGUILayout.FloatField("Distance (z)", it.z);
CrowdClashLevelEditor.cs:468	if (it.type != "gate") it.lane = EditorGUILayout.Popup("Lane", it.lane + 1, Lanes) - 1;
CrowdClashLevelEditor.cs:478	parts[k] = EditorGUILayout.TextField("Panel " + (k + 1), parts[k]);
CrowdClashLevelEditor.cs:481	if (parts.Count > 1 && GUILayout.Button("x", GUILayout.Width(20))) { parts.RemoveAt(k); break; }
CrowdClashLevelEditor.cs:486	if (parts.Count < 3 && GUILayout.Button("+ panel")) parts.Add("+10");
CrowdClashLevelEditor.cs:491	EditorGUILayout.HelpBox("Ops: +N add, xN multiply, -N subtract, /N divide (rounded down).", MessageType.None);
CrowdClashLevelEditor.cs:494	case "enemy": it.count = EditorGUILayout.IntSlider("Squad size", it.count, 1, 400); break;
CrowdClashLevelEditor.cs:495	case "coins": it.count = EditorGUILayout.IntSlider("Coins", it.count, 1, 20); break;
CrowdClashLevelEditor.cs:500	it.ob = kinds[EditorGUILayout.Popup("Obstacle", cur, kinds)];
CrowdClashLevelEditor.cs:502	EditorGUILayout.HelpBox(CCIntro.Name(kk) + " is normally introduced at level " + CCIntro.FirstLevel(kk) + ".", MessageType.Warning);
CrowdClashLevelEditor.cs:507	if (GUILayout.Button("Delete item")) { d.items.RemoveAt(_item); _item = -1; Touch(); }
CrowdClashLevelEditor.cs:553	_genFold = EditorGUILayout.Foldout(_genFold, "Auto Generate (shared runtime generator)", true, EditorStyles.foldoutHeader);
CrowdClashLevelEditor.cs:555	_genCount = EditorGUILayout.IntSlider("Count", _genCount, 1, 50);

`CrowdClashLevelEditor.cs:556`	`_genStart = EditorGUILayout.IntField("Start level (1-based)", Mathf.Max(1, _genStart));`
`CrowdClashLevelEditor.cs:557`	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed);`
`CrowdClashLevelEditor.cs:558`	`_genStartD = EditorGUILayout.Slider("Start difficulty", _genStartD, 0, 1);`
`CrowdClashLevelEditor.cs:559`	`_genEndD = EditorGUILayout.Slider("End difficulty", _genEndD, 0, 1);`
`CrowdClashLevelEditor.cs:560`	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 3, 8);`
`CrowdClashLevelEditor.cs:561`	`_genBoss = EditorGUILayout.Toggle("Boss every 3rd level", _genBoss);`
`CrowdClashLevelEditor.cs:562`	`_genReplace = EditorGUILayout.Toggle("Replace existing range", _genReplace);`
`CrowdClashLevelEditor.cs:566`	`if (GUILayout.Button("Generate " + _genCount + " level(s) at " + _genStart, GUILayout.Height(26))) Generate();`
`CrowdClashLevelEditor.cs:569`	`if (GUILayout.Button("Use start = after last")) _genStart = _file.levels.Count + 1;`
`CrowdClashLevelEditor.cs:570`	`if (GUILayout.Button("Select level as start")) _genStart = _sel + 1;`

CHAPTER 07

Generator, insertion and replacement

Use the actual generation panel and its fields listed below. Set a reproducible seed and a small count, then review the start/index and replace option before clicking Generate. The exact generation implementation is reproduced separately with line references so append, insert and replacement behavior can be distinguished. Save and reopen the data file to verify the intended entry count; runtime generated continuation is not a project-asset save.

Source	Exact current code
`CrowdClashLevelEditor.cs:36`	`bool _genReplace, _genBoss = true;`
`CrowdClashLevelEditor.cs:562`	`_genReplace = EditorGUILayout.Toggle("Replace existing range", _genReplace);`
`CrowdClashLevelEditor.cs:595`	`if (_genReplace && idx < _file.levels.Count) _file.levels[idx] = data;`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/CrowdClash/Resources/crowdclash/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
 "name": "First Steps",
 "theme": 0,
 "width": 4,
 "finish": "stairs",
 "perStep": 2,
 "bossHp": 40,
 "hint": "Drag to steer into the bigger number!",
 "items": [
   {
     "type": "gate",
     "z": 12,
     "ops": "+5|x2",
     "count": 0,
     "ob": "",
     "lane": 0
   },
   {
     "type": "coins",
     "z": 17,
     "ops": "",
     "count": 6,
     "ob": "",
     "lane": 0
   },
   {
     "type": "gate",
     "z": 28,
     "ops": "x3|+10",
     "count": 0,
     "ob": "",
     "lane": 0
   },
   {
     "type": "coins",
     "z": 33,
     "ops": "",
     "count": 6,
     "ob": "",
     "lane": -1
   },
   {
     "type": "gate",
     "z": 44,
     "ops": "+15|x2",
     "count": 0,
     "ob": "",
     "lane": 0
   },
   {
     "type": "coins",
     "z": 49,
     "ops": "",
     "count": 6,
     "ob": "",
     "lane": 1
   },
   {
     "type": "gate",
     "z": 60,
     "ops": "x2|+20",
     "count": 0,
     "ob": "",
     "lane": 0
   }
 ]
}
```

Root keys: levels. Inspect CCLevels.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/CrowdClashGame.cs`. Find `Ink`, currently `#2B2240`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`CrowdClashGame.cs:1166 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }`	`var ct = HzUI.Label("N", coinRow, "+" + baseCoins, 54, Ink); ct.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(200, 60), new Vector2(40, 0));`
`CrowdClashGame.cs:1176 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }`	`var needle = HzUI.Panel("Needle", bar, Ink, 6); needle.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(12, 92), new Vector2(0, 0));`

Source	Exact current code
`CCCrowd.cs:42 / class/static declarations`	`_eyes = CCMat.Get(HzUI.Hex("#20163A"), false, 0.8f);`
`CCWorld.cs:12 / class/static declarations`	`new CCPalette { Name = "Sunset", SkyTop = "#8E7CF0", SkyHorizon = "#FFC7A6", Ground = "#B38AE8", TrackA = "#FFF6EA", TrackB = "#FBE3CF", Rail = "#B98CFF", Sun = "#FFE8D2", Ambient = "#E9C9E8", Pillars = new[] { "#FF9EC4", "#FFC98B", "#A98BFF", "#FFB0A0" } },`
`CCWorld.cs:13 / class/static declarations`	`new CCPalette { Name = "Mint Lagoon", SkyTop = "#4FB6E8", SkyHorizon = "#C8F4EE", Ground = "#4AA9C4", TrackA = "#F4FBF9", TrackB = "#D9F0EC", Rail = "#58B8E8", Sun = "#FFF8EC", Ambient = "#BFE6EA", Pillars = new[] { "#7FD8C4", "#B7F0A0", "#6CB8E8", "#9FE3D6" } },`
`CCWorld.cs:14 / class/static declarations`	`new CCPalette { Name = "Berry Night", SkyTop = "#2C2470", SkyHorizon = "#B07AD8", Ground = "#2F2668", TrackA = "#F0EAFF", TrackB = "#D9CCF8", Rail = "#FF7EB6", Sun = "#FFD8F0", Ambient = "#9E8AD8", Pillars = new[] { "#8A6FE0", "#C66BD8", "#4E7BE0", "#E58BD0" } },`

CCWorld.cs:15 / class/static declarations	new CCPalette { Name = "Lemon Sky", SkyTop = "#FFB547", SkyHorizon = "#FFF1B8", Ground = "#F0A35E", TrackA = "#FFFCEF", TrackB = "#FFEFC2", Rail = "#FF9F43", Sun = "#FFF6DC", Ambient = "#F7DDB0", Pillars = new[] { "#FFC93C", "#FF8E72", "#9AD36A", "#FFD978" } },
CCWorld.cs:16 / class/static declarations	new CCPalette { Name = "Bubblegum", SkyTop = "#FF7EB6", SkyHorizon = "#FFE0EE", Ground = "#E46AA6", TrackA = "#FFF5FA", TrackB = "#FFDDEB", Rail = "#7A6CF0", Sun = "#FFF0F6", Ambient = "#F4C6DC", Pillars = new[] { "#FFB3D1", "#B5A4FF", "#8FE0FF", "#FFD1A1" } },
CCWorld.cs:17 / class/static declarations	new CCPalette { Name = "Matcha", SkyTop = "#5FBF8F", SkyHorizon = "#E9FAD8", Ground = "#5D9A5A", TrackA = "#FBFFF4", TrackB = "#E6F5D6", Rail = "#3FA36B", Sun = "#FFFBEA", Ambient = "#CFE9C4", Pillars = new[] { "#A6E37A", "#FFD36E", "#7CC8A8", "#E9A6D0" } },
CCWorld.cs:211 / void Build()	var white = CCMat.Get("#FFFFFF", false, 0.3f); var dark = CCMat.Get("#2B2240", false, 0.3f);
CCWorld.cs:241 / void BuildGates()	var post = CCMat.Get("#FFFFFF", false, 0.7f);
CCWorld.cs:251 / void BuildGates()	var c = good ? HzUI.Hex("#3AA8FF") : HzUI.Hex("#FF3D63"); c.a = 0.5f;
CCWorld.cs:254 / void BuildGates()	var glow = good ? CCMat.Glow("#59C2FF", 2.6f) : CCMat.Glow("#FF4A70", 2.6f);
CCWorld.cs:282 / void BuildCoins()	var gold = CCMat.Get("#FFC93C", false, 0.88f);
CCWorld.cs:283 / void BuildCoins()	var glow = CCMat.Glow("#FFD45C", 2.2f);
CCWorld.cs:301 / void BuildObstacles()	var dark = CCMat.Get("#3A2D5C", false, 0.45f);
CCWorld.cs:302 / void BuildObstacles()	var metal = CCMat.Get("#D9DCEB", false, 0.9f);
CCWorld.cs:303 / void BuildObstacles()	var danger = CCMat.Get("#FF5D73", false, 0.6f);
CCWorld.cs:304 / void BuildObstacles()	var warnGlow = CCMat.Glow("#FF4A70", 1.8f);
CCWorld.cs:328 / void BuildObstacles()	var warn = HzUI.Hex("#FF4D6A"); warn.a = 0.25f;
CCWorld.cs:366 / void BuildObstacles()	Part("Hole", CCMeshes.Box, CCMat.Get("#1B1233", false, 0f), new Vector3((x0 + x1) / 2, 0.012f, e.Z), new Vector3(x1 - x0, 0.02f, 3.4f), tr, null, false);

`CCWorld.cs:367 / void BuildObstacles()`	`Part("Inner", CCMeshes.Box, CCMat.Get("#0D0820", false, 0f), new Vector3((x0 + x1) / 2, 0.016f, e.Z), new Vector3(x1 - x0 - 0.3f, 0.02f, 3.0f), tr, null, false);`
`CCWorld.cs:374 / void BuildObstacles()`	`var lane = HzUI.Hex("#FF4D6A"); lane.a = 0.16f;`
`CCWorld.cs:417 / void BuildStairs()`	`Part("Landing", CCMeshes.Box, CCMat.Get("#FFFFFF", false, 0.3f), new Vector3(0, topY - 1f, cz), new Vector3(6.4f, 2f, 2.8f), null, null, false);`
`CCWorld.cs:418 / void BuildStairs()`	`Chest = Part("Chest", CCMeshes.Box, CCMat.Get("#FFC13B", false, 0.85f), new Vector3(0, topY + 0.5f, cz), new Vector3(1.6f, 1f, 1.1f));`
`CCWorld.cs:419 / void BuildStairs()`	`Part("Lid", CCMeshes.Box, CCMat.Get("#E58A1F", false, 0.85f), new Vector3(0, 0.62f, 0), new Vector3(1.06f, 0.3f, 1.06f), Chest);`
`CCWorld.cs:420 / void BuildStairs()`	`Part("Lock", CCMeshes.Box, CCMat.Glow("#FFE27A", 2.4f), new Vector3(0, 0.25f, -0.52f), new Vector3(0.2f, 0.3f, 0.05f), Chest, null, false);`
`CCWorld.cs:433 / void BuildBoss()`	`var ringC = HzUI.Hex("#8D5BFF"); ringC.a = 0.25f;`
`CCWorld.cs:437 / void BuildBoss()`	`var kingC = HzUI.Hex("#8D5BFF");`
`CCWorld.cs:439 / void BuildBoss()`	`body.GetComponent<MeshRenderer>().sharedMaterials = new[] { CCMat.Get(kingC, false, 0.7f), CCMat.Get("#20163A", false, 0.8f) };`
`CCWorld.cs:441 / void BuildBoss()`	`var gold = CCMat.Get("#FFC93C", false, 0.92f);`
`CCWorld.cs:447 / void BuildBoss()`	`Part("Gem", CCMeshes.Sphere, CCMat.Glow(k % 2 == 0 ? "#FF4D8D" : "#3AD6FF", 2.4f), new Vector3(Mathf.Cos(a) * 0.95f, 0.05f, Mathf.Sin(a) * 0.95f), Vector3.one * 0.22f, KingCrown, null, false);`
`CCWorld.cs:471 / void BuildScenery(HzRng rng)`	`var stick = CCMat.Get("#FFFFFF", false, 0.5f);`
`CrowdClashGame.cs:23 / class/static declarations`	`static readonly Color PlayerCol = HzUI.Hex("#2EE6A6");`

CrowdClashGame.cs:24 / class/static declarations	static readonly Color EnemyCol = HzUI.Hex("#FF5A6E");
CrowdClashGame.cs:25 / class/static declarations	static readonly Color Ink = HzUI.Hex("#2B2240");
CrowdClashGame.cs:26 / class/static declarations	static readonly Color Gold = HzUI.Hex("#FFC93C");
CrowdClashGame.cs:39 / class/static declarations	new HelperInfo { Id = Helper.Shield, Name = "Shield", Art = "icon_shield", Glyph = "", Unlock = 5, Col = HzUI.Hex("#27C6B0"), Duration = 4.5f, Sfx = "shield",
CrowdClashGame.cs:41 / class/static declarations	new HelperInfo { Id = Helper.Magnet, Name = "Magnet", Art = "icon_magnet", Glyph = "", Unlock = 8, Col = HzUI.Hex("#FF6B6B"), Duration = 7f, Sfx = "magnet",
CrowdClashGame.cs:43 / class/static declarations	new HelperInfo { Id = Helper.Double, Name = "Double Up", Art = "icon_double", Glyph = "", Unlock = 11, Col = HzUI.Hex("#3C8DFF"), Duration = 0f, Sfx = "double_up",
CrowdClashGame.cs:45 / class/static declarations	new HelperInfo { Id = Helper.Frenzy, Name = "Frenzy", Art = "icon_frenzy", Glyph = "", Unlock = 15, Col = HzUI.Hex("#FF8A2B"), Duration = 9f, Sfx = "frenzy",
CrowdClashGame.cs:135 / protected override void Build()	_frenzyMat = CCMat.Get("#FF9A3C", false, 0.75f);
CrowdClashGame.cs:149 / void SetupPostFx()	bloom.tint.Override(HzUI.Hex("#FFF0F6"));
CrowdClashGame.cs:155 / void SetupPostFx()	vg.intensity.Override(0.22f); vg.smoothness.Override(0.45f); vg.color.Override(HzUI.Hex("#2B1640"));
CrowdClashGame.cs:237 / void BuildHud()	var bd = HzUI.Panel("Badge", bg.transform, HzUI.Hex("#FF4D8D"), 26); bd.raycastTarget = false;
CrowdClashGame.cs:283 / void RefreshHelpers()	b.BadgeRt.GetComponent<Image>().color = c > 0 ? HzUI.Hex("#FF4D8D") : HzUI.Hex("#8A7FB0");
CrowdClashGame.cs:345 / void CheckUnlocks()	var p = Popup("New Helper!", h.Name + "\n" + h.Desc, h.Col, ("Got it! (+" + FreeCharges + " free)", HzUI.Hex("#2FBF71"), () => { _guide = h.Id; HzTween.Delay(0.3f, CheckUnlocks); }));

`CrowdClashGame.cs:356 / void ShowUpgradeUnlock(string id, string name, string art, string desc)`	`var p = Popup("Upgrade Unlocked!", name + "\n" + desc, HzUI.Hex("#3C8DFF"), ("Show me", HzUI.Hex("#2FBF71"), () =>`
`CrowdClashGame.cs:408 / void BuildLevelLabels()`	`_bossFill = HzUI.Panel("Hp", bg.transform, HzUI.Hex("#FF4D8D"), 18);`
`CrowdClashGame.cs:464 / void BuildReadyOverlay()`	`var b = HzUI.Panel("BossTag", _ready, HzUI.Hex("#8D5BFF"), 36); b.rectTransform.Place(new Vector2(0.5f, 0.6f), new Vector2(620, 100), new Vector2(40, 120));`
`CrowdClashGame.cs:471 / void BuildReadyOverlay()`	`MakeCard("units", "Start Blobs", "icon_units", HzIcons.Groups, HzUI.Hex("#3C8DFF"), UnitsUnlock, () => UnitsLvl, v => UnitsLvl = v,`
`CrowdClashGame.cs:473 / void BuildReadyOverlay()`	`MakeCard("income", "Income", "icon_income", HzIcons.Coin, HzUI.Hex("#FFA51F"), IncomeUnlock, () => IncomeLvl, v => IncomeLvl = v, null);`
`CrowdClashGame.cs:490 / void MakeCard(string id, string title, string art, string glyph, Color col, int unlockAt, Func<int> get, Action<int> set, Action after)`	`var costBg = HzUI.Panel("Cost", card.transform, HzUI.Hex("#2FBF71"), 26); costBg.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(260, 74), new Vector2(0, 50));`
`CrowdClashGame.cs:506 / void Refresh()`	`costBg.color = free ? HzUI.Hex("#FF4D8D") : Coins >= c ? HzUI.Hex("#2FBF71") : HzUI.Hex("#8A7FB0");`
`CrowdClashGame.cs:597 / void Activate(HelperInfo h)`	`var c = HzUI.Hex("#49E8FF"); c.a = 0.34f;`
`CrowdClashGame.cs:600 / void Activate(HelperInfo h)`	`FloatText("SHIELD!", HzUI.Hex("#27C6B0"));`
`CrowdClashGame.cs:602 / void Activate(HelperInfo h)`	`case Helper.Magnet: _active[1] = h.Duration; FloatText("MAGNET!", HzUI.Hex("#FF6B6B")); break;`
`CrowdClashGame.cs:603 / void Activate(HelperInfo h)`	`case Helper.Double: _doubleArmed = true; FloatText("NEXT GATE ×2!", HzUI.Hex("#3C8DFF")); break;`
`CrowdClashGame.cs:604 / void Activate(HelperInfo h)`	`case Helper.Frenzy: _active[3] = h.Duration; _crowd.BodyOverride = _frenzyMat; FloatText("FRENZY!", HzUI.Hex("#FF8A2B")); break;`

`CrowdClashGame.cs:677 / void PointAt(RectTransform target, string text)`	`var lab = HzUI.Label("Label", lbg.transform, text, 44, HzUI.Hex("#FFE066")); lab.rectTransform.Stretch(10);`
`CrowdClashGame.cs:714 / void ShowBanner(string title, string text)`	`var bg = HzUI.Panel("Bg", _banner, HzUI.Hex("#FF4D8D"), 36); bg.rectTransform.Stretch(); bg.raycastTarget = false;`
`CrowdClashGame.cs:760 / protected override void Update()`	`{ _tutorialShown = true; Toast("Steer into the bigger number!", HzUI.Hex("#FFE066"), 1.6f); }`
`CrowdClashGame.cs:1152 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }`	`p = Popup(_def.Boss ? "Boss Down!" : "Level Complete!", " \n ", HzUI.Hex("#2FBF71"),`
`CrowdClashGame.cs:1153 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }`	`("Claim bonus", HzUI.Hex("#FF8A00"), () =>`
`CrowdClashGame.cs:1158 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }`	`("Collect " + baseCoins, HzUI.Hex("#8A7FB0"), () => Collect(baseCoins)));`
`CrowdClashGame.cs:1162 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }`	`var sub = HzUI.Label("Sub", area, subtitle, 38, HzUI.Hex("#5A4636"), TextAnchor.MiddleCenter, true);`
`CrowdClashGame.cs:1171 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }`	`var c = Meter[i] == 5 ? HzUI.Hex("#FF4D8D") : Meter[i] == 3 ? HzUI.Hex("#FFA51F") : HzUI.Hex("#3C8DFF");`
`CrowdClashGame.cs:1218 / void Lose(bool boss)`	`Popup("So Close!", $"The Jelly King has {_bossHp} HP left.", HzUI.Hex("#8D5BFF"),`
`CrowdClashGame.cs:1219 / void Lose(bool boss)`	`($"+{bonus} blobs & retry (ad)", HzUI.Hex("#FF8A00"), () => HzPlatform.Rewarded(GameId, "boss_rescue", ok => { _rescueBonus = ok ? bonus : 0; LoadLevel(); })),`
`CrowdClashGame.cs:1220 / void Lose(bool boss)`	`("Try Again", HzUI.Hex("#2FBF71"), () => { _rescueBonus = 0; LoadLevel(); }));`

`CrowdClashGame.cs:1223 / void Lose(bool boss)`	`Popup("Crowd Popped!", "Your blobs were wiped out.\nPick bigger gates and dodge traps!", HzUI.Hex("#FF5A6E"),`
`CrowdClashGame.cs:1224 / void Lose(bool boss)`	`("Try Again", HzUI.Hex("#2FBF71"), () => { _rescueBonus = 0; LoadLevel(); }));`
`CrowdClashGame.cs:1233 / string M() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`var p = Popup("Settings", "", HzUI.Hex("#4C9EEB"),`
`CrowdClashGame.cs:1234 / string M() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`(S(), HzUI.Hex("#4C9EEB"), null),`
`CrowdClashGame.cs:1235 / string M() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`(M(), HzUI.Hex("#7B61FF"), null),`
`CrowdClashGame.cs:1236 / string M() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`("Restart", HzUI.Hex("#F5A623"), RestartLevel),`
`CrowdClashGame.cs:1237 / string M() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`("Close", HzUI.Hex("#3DBE6E"), null));`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/CrowdClashGame.cs`, find the displayed literal `"Settings"`, and replace its displayed value with `"Options"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact names/unlocks/free charges: Scripts/CrowdClashGame.cs; effect/help strings: Scripts/CrowdClashGame.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`CCLevels.cs:117 / public static string RowCode(GateOp[] ops)`	`public string Name = "";`
`CCLevels.cs:252 / public static float ObstacleLoss(ObKind k)`	`Number = number, Name = data.name ?? "", Theme = data.theme, HalfW = Clamp(data.width <= 0 ? 4f : data.width, 3f, 5f),`
`CCWorld.cs:12 / class/static declarations`	`new CCPalette { Name = "Sunset", SkyTop = "#8E7CF0", SkyHorizon = "#FFC7A6", Ground = "#B38AE8", TrackA = "#FFF6EA", TrackB = "#FBE3CF", Rail = "#B98CFF", Sun = "#FFE8D2", Ambient = "#E9C9E8", Pillars = new[] { "#FF9EC4", "#FFC98B", "#A98BFF", "#FFB0A0" } },`
`CCWorld.cs:13 / class/static declarations`	`new CCPalette { Name = "Mint Lagoon", SkyTop = "#4FB6E8", SkyHorizon = "#C8F4EE", Ground = "#4AA9C4", TrackA = "#F4FBF9", TrackB = "#D9F0EC", Rail = "#58B8E8", Sun = "#FFF8EC", Ambient = "#BFE6EA", Pillars = new[] { "#7FD8C4", "#B7F0A0", "#6CB8E8", "#9FE3D6" } },`
`CCWorld.cs:14 / class/static declarations`	`new CCPalette { Name = "Berry Night", SkyTop = "#2C2470", SkyHorizon = "#B07AD8", Ground = "#2F2668", TrackA = "#F0EAFF", TrackB = "#D9CCF8", Rail = "#FF7EB6", Sun = "#FFD8F0", Ambient = "#9E8AD8", Pillars = new[] { "#8A6FE0", "#C66BD8", "#4E7BE0", "#E58BD0" } },`

CCWorld.cs:15 / class/static declarations	new CCPalette { Name = "Lemon Sky", SkyTop = "#FFB547", SkyHorizon = "#FFF1B8", Ground = "#F0A35E", TrackA = "#FFFCEF", TrackB = "#FFEFC2", Rail = "#FF9F43", Sun = "#FFF6DC", Ambient = "#F7DDB0", Pillars = new[] { "#FFC93C", "#FF8E72", "#9AD36A", "#FFD978" } },
CCWorld.cs:16 / class/static declarations	new CCPalette { Name = "Bubblegum", SkyTop = "#FF7EB6", SkyHorizon = "#FFE0EE", Ground = "#E46AA6", TrackA = "#FFF5FA", TrackB = "#FFDDEB", Rail = "#7A6CF0", Sun = "#FFF0F6", Ambient = "#F4C6DC", Pillars = new[] { "#FFB3D1", "#B5A4FF", "#8FE0FF", "#FFD1A1" } },
CCWorld.cs:17 / class/static declarations	new CCPalette { Name = "Matcha", SkyTop = "#5FBF8F", SkyHorizon = "#E9FAD8", Ground = "#5D9A5A", TrackA = "#FBFFF4", TrackB = "#E6F5D6", Rail = "#3FA36B", Sun = "#FFFBEA", Ambient = "#CFE9C4", Pillars = new[] { "#A6E37A", "#FFD36E", "#7CC8A8", "#E9A6D0" } },
CrowdClashGame.cs:39 / class/static declarations	new HelperInfo { Id = Helper.Shield, Name = "Shield", Art = "icon_shield", Glyph = "", Unlock = 5, Col = HzUI.Hex("#27C6B0"), Duration = 4.5f, Sfx = "shield",
CrowdClashGame.cs:40 / class/static declarations	Desc = "A jelly bubble protects your crowd from traps for a few seconds." },
CrowdClashGame.cs:41 / class/static declarations	new HelperInfo { Id = Helper.Magnet, Name = "Magnet", Art = "icon_magnet", Glyph = "", Unlock = 8, Col = HzUI.Hex("#FF6B6B"), Duration = 7f, Sfx = "magnet",
CrowdClashGame.cs:42 / class/static declarations	Desc = "Pulls every coin nearby into your crowd." },
CrowdClashGame.cs:43 / class/static declarations	new HelperInfo { Id = Helper.Double, Name = "Double Up", Art = "icon_double", Glyph = "", Unlock = 11, Col = HzUI.Hex("#3C8DFF"), Duration = 0f, Sfx = "double_up",
CrowdClashGame.cs:44 / class/static declarations	Desc = "The next gate you pass gives twice the blobs." },
CrowdClashGame.cs:45 / class/static declarations	new HelperInfo { Id = Helper.Frenzy, Name = "Frenzy", Art = "icon_frenzy", Glyph = "", Unlock = 15, Col = HzUI.Hex("#FF8A2B"), Duration = 9f, Sfx = "frenzy",

`CrowdClashGame.cs:46 / class/static declarations`	`Desc = "Your blobs hit twice as hard in clashes and against the Jelly King." },`
`CrowdClashGame.cs:98 / class/static declarations`	`int Coins { get => HzSave.GetInt("crowdclash_coins", 0); set { HzSave.SetInt("crowdclash_coins", Mathf.Max(0, value)); if (_coinText) _coinText.text = Mathf.Max(0, value).ToString(); } }`
`CrowdClashGame.cs:106 / int Charges(Helper h) => HzSave.GetInt(Key("ch_" + h), 0);`	`int Charges(Helper h) => HzSave.GetInt(Key("ch_" + h), 0);`
`CrowdClashGame.cs:107 / void SetCharges(Helper h, int v) { HzSave.SetInt(Key("ch_" + h), Mathf.Max(0, v)); RefreshHelpers(); }`	`void SetCharges(Helper h, int v) { HzSave.SetInt(Key("ch_" + h), Mathf.Max(0, v)); RefreshHelpers(); }`
`CrowdClashGame.cs:117 / protected override void Build()`	`CCLevels.LoadJson(ta ? ta.text : "");`
`CrowdClashGame.cs:161 / void ApplyTheme(CCPalette pal)`	`_sun.color = sun; _sun.intensity = pal.Name == "Berry Night" ? 1.05f : 1.3f;`
`CrowdClashGame.cs:184 / void BuildHud()`	`_levelText = HzUI.Label("Level", _top, "LEVEL 1", 54, Color.white);`
`CrowdClashGame.cs:200 / void BuildHud()`	`_coinText = HzUI.Label("N", pill.transform, Coins.ToString(), 48, Color.white, TextAnchor.MiddleRight);`
`CrowdClashGame.cs:209 / void BuildHud()`	`_bar = HzUI.Rect("Helpers", _hud);`
`CrowdClashGame.cs:235 / void BuildHud()`	`var nm = HzUI.Label("Name", bg.transform, h.Name.ToUpperInvariant(), 26, Color.white); nm.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(170, 34), new Vector2(0, -6));`
`CrowdClashGame.cs:240 / void BuildHud()`	`hb.Badge = HzUI.Label("N", bd.transform, "0", 36, Color.white); hb.Badge.rectTransform.Stretch();`
`CrowdClashGame.cs:245 / void BuildHud()`	`hb.LockText = HzUI.Label("Lv", lk.transform, "LV " + h.Unlock, 32, Color.white); hb.LockText.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(150, 40), new Vector2(0, 24));`
`CrowdClashGame.cs:282 / void RefreshHelpers()`	`b.Badge.text = c > 0 ? c.ToString() : "AD";`
`CrowdClashGame.cs:319 / void LoadLevel()`	`_levelText.text = (_def.Boss ? "BOSS " : "") + "LEVEL " + Level;`

`CrowdClashGame.cs:345 / void CheckUnlocks()`	`var p = Popup("New Helper!", h.Name + "\n" + h.Desc, h.Col, ("Got it! (+" + FreeCharges + " free)", HzUI.Hex("#2FBF71"), () => { _guide = h.Id; HzTween.Delay(0.3f, CheckUnlocks); }));`
`CrowdClashGame.cs:356 / void ShowUpgradeUnlock(string id, string name, string art, string desc)`	`var p = Popup("Upgrade Unlocked!", name + "\n" + desc, HzUI.Hex("#3C8DFF"), ("Show me", HzUI.Hex("#2FBF71"), () =>`
`CrowdClashGame.cs:400 / void BuildLevelLabels()`	`var l = AddLabel("×" + m.ToString("0.0", CultureInfo.InvariantCulture), 70, Color.white, () => p + Vector3.up * 0.2f, 12f);`
`CrowdClashGame.cs:406 / void BuildLevelLabels()`	`_bossBar = AddLabel("", 10, Color.white, () => _world.King.position + Vector3.up * 5.6f, 16f);`
`CrowdClashGame.cs:410 / void BuildLevelLabels()`	`_bossText = HzUI.Label("HpT", bg.transform, _bossHp.ToString(), 40, Color.white); _bossText.rectTransform.Stretch();`
`CrowdClashGame.cs:423 / void BuildLevelLabels()`	`var t = HzUI.Label("T", inner, text, size, c); t.rectTransform.Stretch(); t.horizontalOverflow = HorizontalWrapMode.Overflow;`
`CrowdClashGame.cs:436 / void BuildLevelLabels()`	`var t = HzUI.Label("T", rt, "0", 50, Color.white); t.rectTransform.Stretch(); t.AddOutline(new Color(0, 0, 0, 0.25f), 2);`
`CrowdClashGame.cs:450 / void BuildReadyOverlay()`	`_hintText = HzUI.Label("Hint", _ready, hint, IsPortrait ? 48 : 52, Color.white);`
`CrowdClashGame.cs:466 / void BuildReadyOverlay()`	`var bt = HzUI.Label("T", b.transform, "JELLY KING AHEAD!", 48, Color.white); bt.rectTransform.Stretch(); bt.rectTransform.offsetMin = new Vector2(70, 0);`
`CrowdClashGame.cs:486 / void MakeCard(string id, string title, string art, string glyph, Color col, int unlockAt, Func<int> get, Action<int> set, Action after)`	`var lvl = HzUI.Label("Lvl", card.transform, "", 34, Color.white, TextAnchor.UpperLeft); lvl.rectTransform.Place(new Vector2(0, 1), new Vector2(200, 50), new Vector2(118, -36)); lvl.AddOutline(new Color(0, 0, 0, 0.35f), 2);`
`CrowdClashGame.cs:488 / void MakeCard(string id, string title, string art, string glyph, Color col, int unlockAt, Func<int> get, Action<int> set, Action after)`	`var tt = HzUI.Label("Title", card.transform, title, 40, Color.white); tt.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(300, 56), new Vector2(0, 112));`

`CrowdClashGame.cs:492 / void MakeCard(string id, string title, string art, string glyph, Color col, int unlockAt, Func<int> get, Action<int> set, Action after)`	`var cost = HzUI.Label("C", costBg.transform, "", 40, Color.white); cost.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(190, 70), new Vector2(28, 0));`
`CrowdClashGame.cs:495 / void MakeCard(string id, string title, string art, string glyph, Color col, int unlockAt, Func<int> get, Action<int> set, Action after)`	`var lt = HzUI.Label("Lv", lockP.transform, "UNLOCKS AT\nLEVEL " + unlockAt, 34, Color.white); lt.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(280, 90), new Vector2(0, 60));`
`CrowdClashGame.cs:501 / void Refresh()`	`lvl.text = "LVL " + l;`
`CrowdClashGame.cs:504 / void Refresh()`	`cost.text = free ? "FREE" : Coins >= c ? c.ToString() : "AD";`
`CrowdClashGame.cs:511 / void Refresh()`	`if (!UpgradeUnlocked(unlockAt)) { HzAudio.Play("wrong", 0.6f); HzTween.Shake(card.transform, 14, 0.3f); Toast("Unlocks at level " + unlockAt, Color.white, 1.2f); return; }`
`CrowdClashGame.cs:569 / void OnHelper(HelperInfo h)`	`if (!Unlocked(h)) { HzAudio.Play("wrong", 0.6f); HzTween.Shake(_hb[(int)h.Id].Rt, 14, 0.3f); Toast(h.Name + " unlocks at level " + h.Unlock, Color.white, 1.3f); return; }`
`CrowdClashGame.cs:570 / void OnHelper(HelperInfo h)`	`if (_phase == Phase.Ready) { Toast("Start running first!", Color.white, 1.1f); HzAudio.Play("wrong", 0.5f); return; }`
`CrowdClashGame.cs:572 / void OnHelper(HelperInfo h)`	`if (IsActive(h.Id)) { Toast(h.Name + " is already active", Color.white, 1f); return; }`
`CrowdClashGame.cs:602 / void Activate(HelperInfo h)`	`case Helper.Magnet: _active[1] = h.Duration; FloatText("MAGNET!", HzUI.Hex("#FF6B6B")); break;`
`CrowdClashGame.cs:603 / void Activate(HelperInfo h)`	`case Helper.Double: _doubleArmed = true; FloatText("NEXT GATE ×2!", HzUI.Hex("#3C8DFF")); break;`
`CrowdClashGame.cs:604 / void Activate(HelperInfo h)`	`case Helper.Frenzy: _active[3] = h.Duration; _crowd.BodyOverride = _frenzyMat; FloatText("FRENZY!", HzUI.Hex("#FF8A2B")); break;`
`CrowdClashGame.cs:666 / void UpdateGuide()`	`PointAt(_hb[(int)g].Rt, "Tap " + Helpers[(int)g].Name.ToUpperInvariant() + "!");`

CrowdClashGame.cs:677 / void PointAt(RectTransform target, string text)	var lab = HzUI.Label("Label", lbg.transform, text, 44, HzUI.Hex("#FFE066")); lab.rectTransform.Stretch(10);
CrowdClashGame.cs:716 / void ShowBanner(string title, string text)	var t = HzUI.Label("T", bg.transform, title, 44, Color.white); t.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(_banner.sizeDelta.x - 40, 56), new Vector2(0, -32));
CrowdClashGame.cs:717 / void ShowBanner(string title, string text)	var s = HzUI.Label("S", bg.transform, text, 32, Color.white, TextAnchor.MiddleCenter, false); s.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(_banner.sizeDelta.x - 40, 48), new Vector2(0, 30));
CrowdClashGame.cs:760 / protected override void Update()	{ _tutorialShown = true; Toast("Steer into the bigger number!", HzUI.Hex("#FFE066"), 1.6f); }
CrowdClashGame.cs:1152 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }	p = Popup(_def.Boss ? "Boss Down!" : "Level Complete!", " \n ", HzUI.Hex("#2FBF71"),
CrowdClashGame.cs:1160 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }	if (p.Message) p.Message.text = "";
CrowdClashGame.cs:1162 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }	var sub = HzUI.Label("Sub", area, subtitle, 38, HzUI.Hex("#5A4636"), TextAnchor.MiddleCenter, true);
CrowdClashGame.cs:1166 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }	var ct = HzUI.Label("N", coinRow, "+" + baseCoins, 54, Ink); ct.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(200, 60), new Vector2(40, 0));
CrowdClashGame.cs:1173 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }	var st = HzUI.Label("T", s.transform, "×" + Meter[i], 40, Color.white); st.rectTransform.Stretch();
CrowdClashGame.cs:1188 / int MeterIndex() { float s = Mathf.PingPong((Time.time - t0) * 1.6f, 1f); return Mathf.Clamp(Mathf.FloorToInt(s * Meter.Length), 0, Meter.Length - 1); }	if (claim) claim.SetText("Claim ×" + Meter[mi] + " (ad)");
CrowdClashGame.cs:1218 / void Lose(bool boss)	Popup("So Close!", $"The Jelly King has {_bossHp} HP left.", HzUI.Hex("#8D5BFF"),

`CrowdClashGame.cs:1223 / void Lose(bool boss)`	`Popup("Crowd Popped!", "Your blobs were wiped out.\nPick bigger gates and dodge traps!", HzUI.Hex("#FF5A6E"),`
`CrowdClashGame.cs:1231 / string S() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`string S() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`CrowdClashGame.cs:1232 / string M() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`string M() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`
`CrowdClashGame.cs:1233 / string M() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`var p = Popup("Settings", "", HzUI.Hex("#4C9EEB"),`
`CrowdClashGame.cs:1347 / public override string DebugCommand(string cmd)`	`foreach (var h in Helpers) ch.Append(h.Id).Append(':').Append(Unlocked(h) ? Charges(h.Id).ToString() : "locked").Append(IsActive(h.Id) ? "*" : "").Append(' ');`
`CrowdClashGame.cs:1371 / public override string DebugCommand(string cmd)`	`if (a.Length > 1 && Enum.TryParse<Helper>(a[1], true, out var hh)) { OnHelper(Helpers[(int)hh]); return "used " + hh; }`
`CrowdClashGame.cs:1376 / public override string DebugCommand(string cmd)`	`foreach (var h in Helpers) { SetFlag("unl_" + h.Id, false); SetFlag("guide_" + h.Id, false); SetCharges(h.Id, 0); }`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/crowdclash through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/crowdclash/Sfx. Preserve each actual load key and importer settings when replacing sounds. This game has no supplied music loop under its Resources Music folder; do not assume that a Music toggle is proof of a bundled track. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.crowdclash`, version, scene orientation and signing. The build command writes `Builds/Android/crowdclash.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/crowdclash/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.